Task 1: Problem Definition and Dataset Selection

- Define a meaningful real-world problem suitable for data-driven analysis.
- Explain the motivation, background, stakeholders, and practical relevance of the selected problem.
- Select an appropriate open-source or publicly accessible dataset.
- Ensure the dataset contains tabular, alpha-numeric data; image datasets are not permitted.
- Ensure the dataset contains more than 20 and fewer than 300 features at the initial data preparation stage.
- Provide the dataset source, access date, licence or terms of use, and relevant documentation.
- Obtain approval from the course lecturers for the selected problem and dataset.

Problem Statement:

Credit card fraud to this day presents a persistent and costly issue for financial systems around the world. As digital and card not present transitions (online shopping, phone orders, subscription payments) have become the norm, chances for fraudulent activity has grown alongside them, and traditional rule-based detection systems (criteria list, blacklists, human review) struggle to keep up with both the ever-growing number of transactions and the constant adaptation of hackers. This project addresses a binary classification problem: given a set of transaction-level features, predict whether an individual credit card transaction is fraudulent or legitimate. The goal is to demonstrate how a data-driven, machine learning-based approach can support automated, scalable fraud screening.

Motivation and Background:

Global financial systems process millions of transactions on the daily, making manual review of every transaction time consuming and prone to human error. Machine learning offers a way to automatically flag suspicious transactions in sub real time by learning patterns from data rather than relying on static rules. However, fraud detection poses a difficult modelling challenge: fraudulent transactions are extremely rare compared to legitimate ones, meaning the dataset is highly imbalanced, and the cost of different types of errors is uneven. A missed fraudulent transaction (false negative) results in financial lost, whilst an incorrectly flagged legitimate transaction (false positive) inconveniences a customer and can harm the customer-company trust. This makes the

problem a strong contender for exploring not just model accuracy, but metrics that keep up with real-world costs, such as precision, recall, the trade off, and the constant attempt to keep up with an ever-changing environment.

Stakeholders:

- **Cardholders** – Directly affected by monetary loss and the inconvenience of false declines on legitimate purchases
- **Banks** – Hold financial liability for confirmed fraudulent transactions and face compliance obligations as well as risk reputational damage from improper fraud handling
- **Merchants** – Soak up costs through chargebacks and loss of goods or services already sourced
- **Payment networks** – Rely heavily on system-wide trust in the network to maintain usage

Practical Relevance:

Effective fraud detection systems reduce financial losses in payments whilst maintaining customer trust and convenience. The connection between catching fraud and avoiding false alarms is integral to designing a proper system. A model that's tuned solely for accuracy can perform misleadingly well by assuming legitimate for almost every transaction, given how rare fraud is in the data. This'll grab more attention to evaluation strategy in the later stages of the project and shows an actual industry relevant trade off that data driven fraud systems must navigate in practice.

Task 2: Data Acquisition, Inspection, and Documentation

- Load the dataset using Python. Other programming languages are not permitted.
- Describe the dataset structure, including the number of records, number of features, data types, variable meanings, etc.
- Inspect data types, value ranges, unique values, missing values, duplicate records, inconsistent entries, and invalid values.
- Document assumptions made during data interpretation and preparation.

Task 3: Data Cleansing and Transformation

- Identify and address missing values, duplicate records, invalid values, inconsistent categories, incorrect data types, and formatting issues.
- Investigate potential outliers and explain whether they are retained, transformed, capped, or removed.
- Apply appropriate transformations such as standardising formats, unit conversion, categorical encoding, scaling, aggregation, reshaping, or data-type conversion where relevant.
- Ensure that all data-cleaning and transformation decisions are clearly explained and justified.

Task 4: Exploratory Data Analysis and Visualisation

- Conduct descriptive analysis of relevant numerical and categorical variables.
- Produce appropriate visualisations such as histograms, boxplots, bar charts, scatter plots, pair plots, missing-value plots, or correlation heatmaps.
- Analyse feature distributions, missingness patterns, class or target distribution, relationships between variables, possible data imbalance, and unusual patterns.
- Summarise key data insights that help explain the structure, quality, and usefulness of the dataset.

However, each student must complete the practical and written work individually. Each student must independently:

- write and run their own Python code;
- perform their own data cleaning and analysis;
- create their own visualisations;
- conduct and interpret their own statistical tests;
- explain and justify their own decisions

Discussion within the group is encouraged, but students must not copy or share completed code, written explanations, interpretations or assessment responses.